COS 214 – TEAM OBSIDIAN

Team Mates:

- Ngeletshedzo Mutwanamba (Team Leader) – u25039769
- Amira Ajanaku – u25111699
- Nare Maela - u24607917
- Nontokozo Madela - u25677404
- Manqoba Mashego – u24589137
- Giselle Mpye - u25622103
- Shepherd Nzimande - u25066120

Github Repo:

https://github.com/ngeletech/COS214-PROJ

Task 1: Research Findings on Workflow Management Systems

What is a workflow? A workflow is basically a business process that has been automated, where documents, information or tasks get passed between people following a set of rules. A workflow management system (WfMS) is the software that sets these workflows up and runs them, and the workflow engine is the part that does the actual running. Business process management (BPM) is the bigger picture, because it also covers analysing processes and how work is organised, not just automating it (Workflow Management Coalition, 1994).

Definition vs instance. The definition is the blueprint: the steps in a process and the rules for moving between them. An instance is one running copy of that blueprint. Lots of instances can run at the same time from one model, and each one deals with its own case, so each needs its own state and data (Workflow Management Coalition, 1994) (van der Aalst, 2013).

Activities and sub-processes. An activity is one step in the process, and when a person must do it, a work item shows up on their worklist. BPMN 2.0 splits tasks into user tasks (a human does it) and service tasks (a system does it). It also has sub-processes, which are activities with their own flow inside, and call activities, which reuse a process that is defined somewhere else (Object Management Group, 2011).

Routing. The workflow patterns research lists the basic ways to route work: sequence, parallel split (AND), synchronisation, exclusive choice (XOR), simple merge, multi-choice and loops. Different products support different ones, so it makes sense to keep routing flexible and not hard-code it (van der Aalst, et al., 2003).

Participants and assigning work. Engines look after worklists and who is signed in. The BPM survey describes people using roles and organisational units, including rules like the four-eyes principle, so the same person cannot do two dependent tasks. Work can reach people in two ways: the system pushes it to a worker, or the worker pulls it from the system (Workflow Management Coalition, 1994; van der Aalst, 2013; Russell et al., 2005) (Russell, et al., 2005).

Events, failures and external systems. In BPMN, boundary events sit on the edge of an activity and react while it is running. There are timer, message, error and escalation events. Compensation can undo activities that already finished if something goes wrong. The WfMC model also has separate interfaces for calling outside applications and talking to other workflow systems (Object Management Group, 2011).

History and changing definitions. Everything that happens gets saved in event logs, where each event points to an activity and a case, and process mining uses these logs to find out how processes really run. The WfMC treats audit as part of monitoring. To see how a real product deals with a definition changing while work is still running, we looked at the documentation of Camunda, a company that makes a workflow engine. In Camunda, a changed definition is saved as a new version. Running instances stay on the old version, new ones use the latest, and teams can choose to move running instances over without losing their history (van der Aalst, 2013) (Workflow Management Coalition, 1994) (Camunda, n.d).

What this means for TIVIDY. Our takeaway is that the engine should only know generic things like activities, instances, states, participants and events. Anything that changes between processes (the structure and versions, how work is grouped, routing, assignment and approval rules, events and integrations, and reports and rollback) should be able to change without touching the rest.

Task 2: Defining the scenarios and system

Scenario 1

Overview

In a meal plan order-to-delivery shopping system that offers custom meal packages for customers; Breakfast, Lunch and Dinner. It manages customer orders from payment and coupon discounts, through kitchen cooking and packaging and finally courier handoff and delivery. The goal is to deliver the meal package to the customer successfully while keeping an accurate record of every step.

Industry: Retail Food and Catering Logistics
The goal of this system is to allow users to customize meal packages (e.g. Breakfast, Lunch and Dinner), apply discounts coupons, prepare/package food in the kitchen and route deliveries through regional courier APIs.
Input data include: a WorkItemSpec list, Customer Delivery Regions, Coupon Codes and Meal Types.
The result to expect is a delivered meal package, that is recorded in the audit log, or is in a clean rollback and can be refunded if a unresolvable exception occurs.

Generic Engine Abstraction vs. Scenario Roles

Engine Concept	Generic Class Name	Scenario Mapped Value / Instance
Unified System Boundary	TividyFacade	Entry point used by the UI to define, start, track, and manage orders.
Encapsulated Operations	StartInstanceCommand, CancelCommand, ChangeDataCommand	Invoked via CommandInvoker to execute or undo workflow actions
Task Execution Skeleton	Task (UserTask, ServiceTask, ApprovalTask, ScriptTask)	Skeleton algorithm in Task::execute() calling doWork() for prep, approvals, and API calls.
History & Tree Navigation	DepthFirstIterator, AvailableItemIterator, AuditEntryIterator	Iterates over composite order trees and audit event logs.
Participants	Participant	KitchenStaff, Supervisor, BranchManager, CourierAgent
Routing Strategy	RegionBasedAssignment	Maps customer postal code to CourierA, CourierB or CourierC
Escalation Chain	FrontlineHandler → SupervisorHandler → ManagerHandler	KitchenStaffHandler → SupervisorHandler → BranchManagerHandler
Work Items	UserTask, ServiceTask, ApprovalTask, ScriptTask	PrepMealStep, PackagingTask, ManagerSignoff, UpdateOrderStatus
Composite Groups	SequentialGroup, ParallelGroup	FullDayPackage (groups Breakfast, Lunch, Dinner in parallel)

Detailed Execution Paths

Path A: Successful Completion Path

1. **Facade Entry & Definition Creation (Facade + Builder + Factory Method):**
    - The client application calls TividyFacade::defineWorkflow(). WorkflowDesigner uses StandardDefinitionBuilder to build the complete blueprint (ValidateCoupon → CookMeals → PackageFood → CourierDelivery).
    - When an order starts via TividyFacade::startWorkflow(), WorkflowEngine uses WorkItemFactory to create independent task objects.
2. **Encapsulated Trigger (Command):**
    - The UI issues a StartInstanceCommand through CommandInvoker to begin order execution.
3. **Discount Wrapping (Decorator):**
    - Base order cost is wrapped with a CouponDecorator to dynamically modify order pricing.
4. **Parallel Cooking & Tree Traversal (Composite + Iterator):**
    - The engine executes a FullDayPackage (ParallelGroup).
    - DepthFirstIterator and AvailableItemIterator navigate the task tree to find ready tasks. Breakfast, Lunch, and Dinner tasks run in parallel.
5. **Standardized Task Execution Pipeline (Template Method):**
    - Each work item executes via Task::execute(). The skeleton algorithm checks readiness (checkReady()), transitions status (STARTED), executes concrete work (doWork()), notifies observers, and marks completed.
    - UserTask runs manual prep, ScriptTask updates internal flags, and ApprovalTask handles kitchen manager signoffs.
6. **State Transition & Alerting (State + Observer):**
    - When all meals finish cooking, the instance transitions from PreparingState to PackagingState. WorkflowEventPublisher notifies SmsNotifier (SMS to customer) and SupportDashboardListener.
7. **Courier Assignment & Dispatch (Strategy + Adapter):**
    - AssignmentService uses RegionBasedAssignment (Strategy) to pick a courier based on delivery region.
    - The ServiceTask hands payload data to the selected CourierAdapter (Adapter), translating requests for CourierAApi, CourierBApi, or CourierCApi.
8. **Completion & History Inspection (Iterator):**
    - Instance moves to DeliveredState. AuditLogger appends execution logs, which can be inspected sequentially using AuditEntryIterator.

Path B: Exception, Escalation, and Rollback Path

1. **Out of Stock / Ingredient Failure (Template Method):**
    - During preparation inside PrepMealStep (UserTask::doWork()), kitchen staff encounters missing ingredients and raises an Issue.
2. **Chain of Responsibility Escalation:**
    - The Issue passes to KitchenStaffHandler (severity low).
    - Unresolved, it escalates to SupervisorHandler.
    - Unresolved, it escalates to BranchManagerHandler.
3. **Unresolvable Issue / Customer Cancellation (Command):**
    - If the branch manager cannot resolve the shortage or the customer cancels, a CancelCommand is executed via CommandInvoker.
4. **State Guard Enforcement (State):**

- The engine checks canCancel() on PreparingState (valid). If the order was already in InTransitState, canCancel() returns false and throws std::logic_error.

5. **Undo State (Memento + Command):**
   - RollbackManager retrieves the InstanceSnapshot saved before cooking.
   - Alternatively, calling TividyFacade::undoLast() triggers command undoing to restore the previous state without losing history logs.
6. **Failure State & Log Inspection (Iterator):**
   - Instance moves to CancelledState / FailedState, alerts support, and AuditEntryIterator displays the exact failure sequence in reports.

Scenario 2

Overview

When a customer orders home fiber internet or a mobile phone plan, the telecom company must process their account, check credit, send a technician to install the line, configure the router, and activate their service. The goal is to get the customer connected smoothly while logging every step..

Industry / Domain: Telecommunications & IT Infrastructure
Process Name: Fiber & Mobile Network Service Order-to-Activation
Business Goal: Manage new customer onboarding for high-speed fiber internet or corporate mobile data subscriptions. The workflow guides the request from account verification, package bundling, physical line installation, IP allocation, to SIM/Router activation and billing initiation.
Input Data: Service package choice, customer location/address, coupon or priority add-on, and device details (router/SIM card).
Expected Outcome: An active internet connection logged in the audit history, or a clean cancellation/refund if the installation cannot be completed.

Generic Engine Abstraction vs. Telecom Roles

Engine Concept	Generic Class Name	Scenario Role / Example
Main System Entry	TividyFacade	The online portal or agent dashboard used to place and manage telecom orders.
System Actions	StartInstanceCommand, CancelCommand, ChangeDataCommand	Triggers the setup, changes speed tiers, or cancels an order.
Task Execution	Task (UserTask, ServiceTask, ApprovalTask, ScriptTask)	Runs physical line installation, manager signoffs, automated system checks, and database updates.
Tree & History Navigation	DepthFirstIterator, AvailableItemIterator, AuditEntryIterator	Checks step-by-step progress and reads activation history logs.
People Involved	Participant	InstallationTechnician, CreditOfficer, NetworkEngineer, SystemBot.
Routing Strategy	RegionBasedAssignment	Selects local field teams or server gateways based on customer location.
Problem Escalation	FrontlineHandler → SupervisorHandler → ManagerHandler	SupportAgentHandler → FieldSupervisorHandler → OperationsManagerHandler.
Combined Packages	SequentialGroup, ParallelGroup	DoublePlayBundle (installs internet line and sets up WiFi router at the same time).

Detailed Execution Paths

Path A: Successful Installation & Activation Path

1. **Placing the Order (Facade + Builder + Factory Method):**
    - The customer places an order on the website (TividyFacade::startWorkflow()).
    - The system uses StandardDefinitionBuilder to assemble the steps (CreditCheck → InstallFiberLine → ConfigureRouter → ActivateAccount).

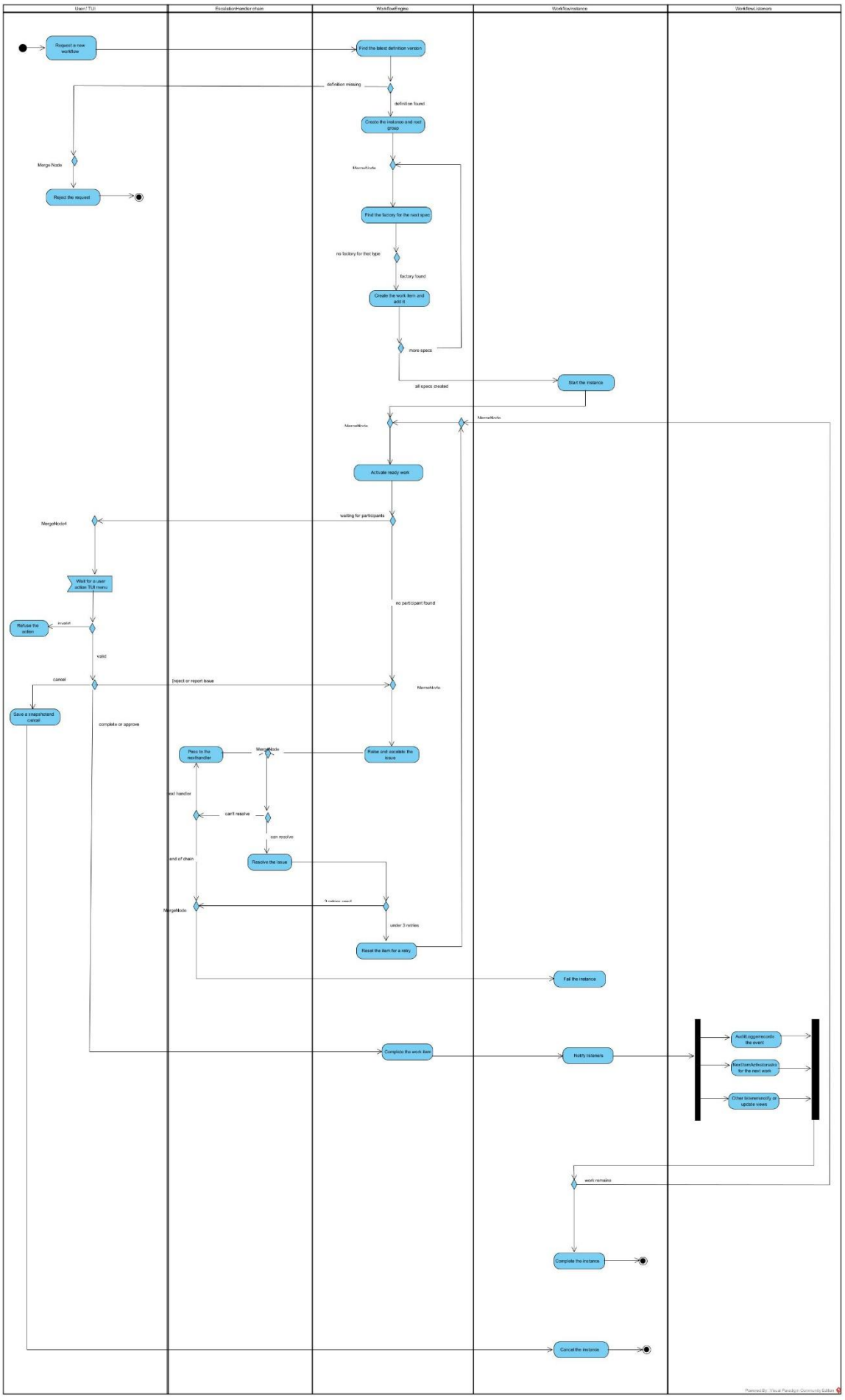
    - Independent task objects are created specifically for this customer via WorkItemFactory.
2. **Starting Setup (Command):**
    - A StartInstanceCommand runs to start processing the order.
3. **Adding Speed Add-ons (Decorator):**
    - If the customer adds a speed boost or static IP, a PriorityDecorator or StaticIPDecorator wraps the base plan to increase speed or update settings.
4. **Installing Line & Router in Parallel (Composite + Iterator):**
    - The system runs a DoublePlayBundle (ParallelGroup).
    - The engine uses DepthFirstIterator to find ready tasks. The physical line setup and router pre-configuration happen simultaneously.
5. **Executing Each Task (Template Method):**
    - Every step follows the engine's Task::execute() workflow.
    - UserTask: A field technician completes physical line installation.
    - ApprovalTask: A credit officer approves the monthly payment plan.
    - ScriptTask: An automated script sets AccountStatus = ACTIVE in the system.
6. **Updating Status & Sending Alerts (State + Observer):**
    - Once installed, the order moves from PreparingState to PackagingState (Final Setup).
    - An observer sends an SMS notification with account login details to the customer.
7. **Activating Network Connection (Strategy + Adapter):**
    - AssignmentService uses RegionBasedAssignment (Strategy) to choose the local network hub based on the customer's area.
    - A ServiceTask uses a NetworkAdapter (Adapter) to communicate with third-party network hardware to switch on internet access.
8. **Finalizing Order (Iterator):**
    - Order status changes to DeliveredState (Connected). AuditLogger saves every completed step, which can be viewed via AuditEntryIterator.

Path B: Exception, Problem Escalation, and Cancellation Path

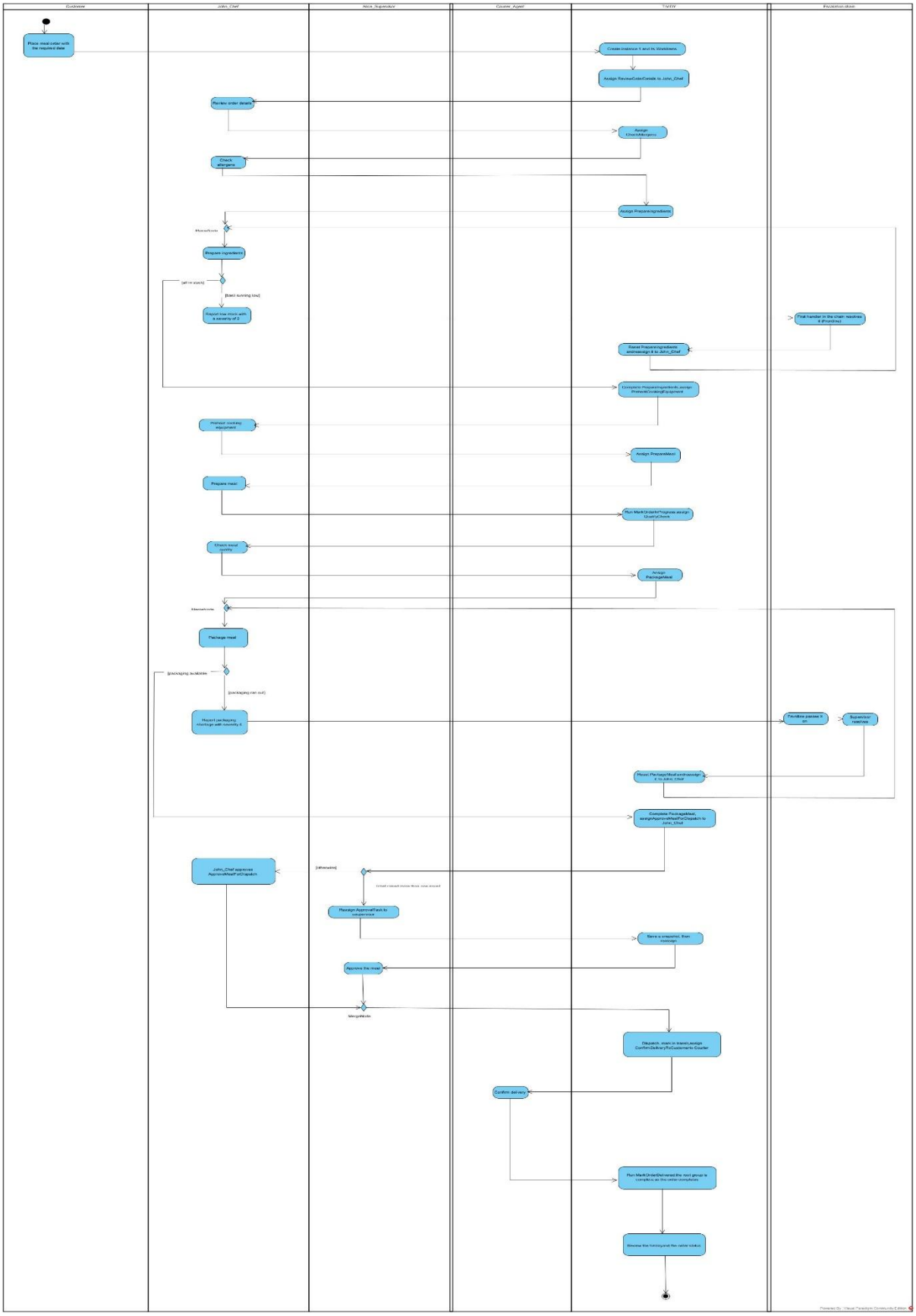
1. **Installation Problem Arises (Template Method):**
    - During physical installation (UserTask::doWork()), the technician discovers damaged street cabling and raises an Issue.
2. **Escalating the Problem (Chain of Responsibility):**
    - The issue goes to the SupportAgentHandler.
    - If unsolved, it escalates to the FieldSupervisorHandler.
    - If still unsolved, it reaches the OperationsManagerHandler.
3. **Cancelling the Unresolvable Order (Command):**
    - If repairs take too long and the customer wants to cancel, a CancelCommand is issued.
4. **Checking State Safety Rules (State):**
    - The engine verifies canCancel() on PreparingState (Allowed). If the service was already fully active (DeliveredState), cancelling is blocked and requires a formal disconnect request instead.
5. **Rolling Back Reserved Settings (Memento + Command):**
    - RollbackManager restores the InstanceSnapshot created before setup began.
    - Or TividyFacade::undoLast() reverses recent changes to release reserved IP addresses/router details cleanly.
6. **Logging the Cancellation (Iterator):**
    - The order moves to CancelledState / FailedState. AuditEntryIterator displays the exact history log showing where and why the installation stopped.

Task 3: UML Activity Diagrams

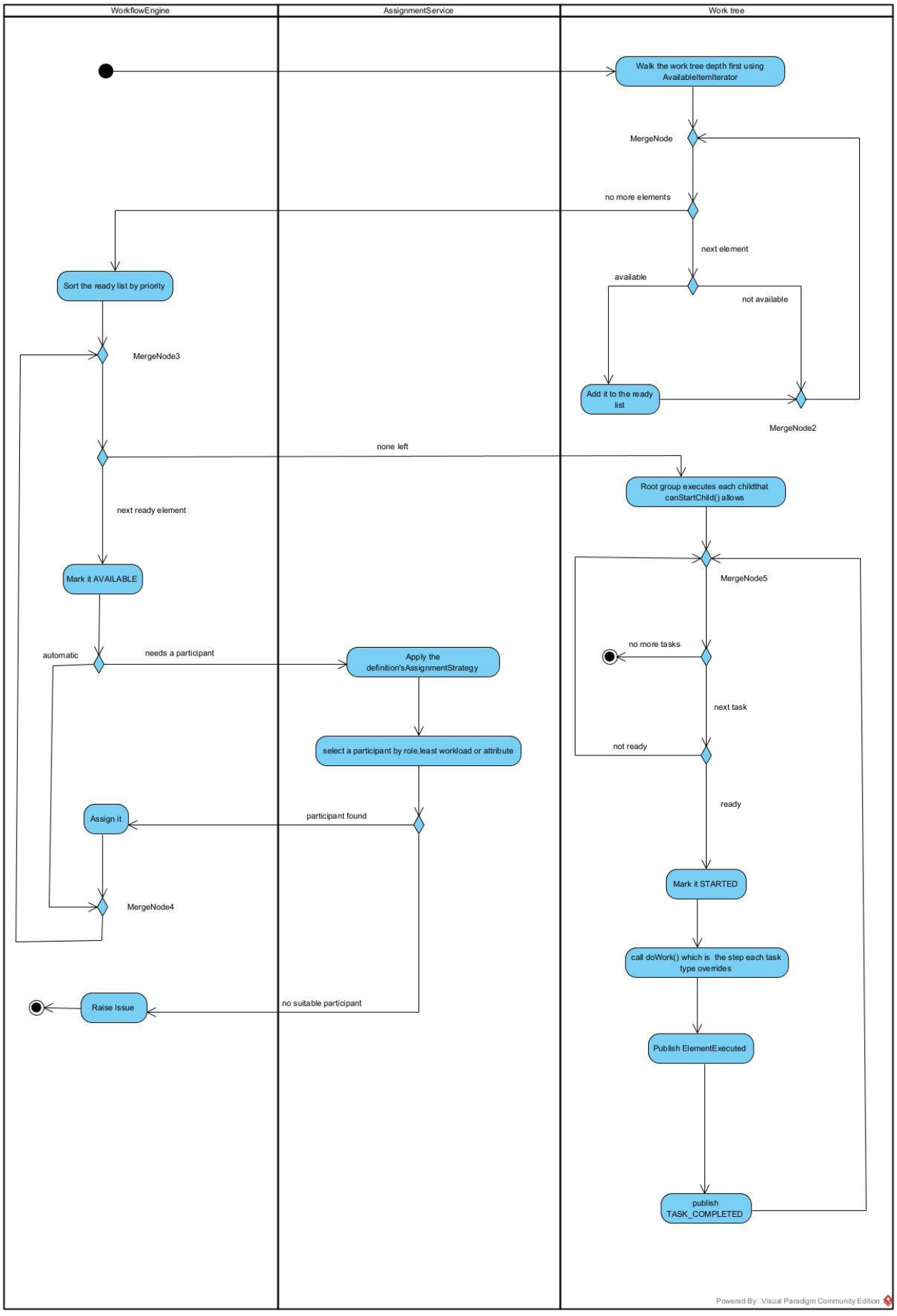
UML Activity Diagram 1: Generic workflow execution



UML Activity Diagram 2: Generic workflow showing escalation



UML Activity Diagram 3: Activate available work



Task 4: Design Patterns

Each pattern's problem is described using our meal-plan scenario as the worked example. All participating classes are generic workflow classes, and the scenario is applied to them through the workflow definition's configuration.

1. Builder

Problem: A meal-plan definition is assembled in several steps (validate coupon → prepare meals → package → courier delivery), with routing rules and an escalation chain attached. It must be complete and validated before any customer order runs from it, and it must not change under running orders.

Participants:

GoF Participants	Class Names
Builder	WorkflowDefinitionBuilder
ConcreteBuilder	StandardDefinitionBuilder
Director	WorkflowDesigner
Product	WorkflowDefinition

Collaboration: The WorkflowDesigner directs the WorkflowDefinitionBuilder step by step, telling it which stages, work items, assignment rules and escalation chain to include. StandardDefinitionBuilder records each step into the definition under construction. Once the designer finishes, the builder checks that the definition is complete and hands back a versioned WorkflowDefinition.

Variation: The steps, their order and the rules attached to them (assignment strategy, escalation chain) differ for every kind of process. A new version of the definition is produced whenever any of them changes.

Why is it appropriate: The Builder is the construction side of the definition/instance split and the only place where a definition is created. By the time a WorkflowDefinition reaches the engine it is complete, validated and read-only, so later edits cannot disturb orders already running.

2. Factory Method

Problem: Every customer order is its own workflow instance and needs fresh task objects (these are the meal tasks and packaging for each customers order in our scenario) created from the shared definition. A change to one instance must never affect the other.

Participants:

GoF Participants	Class Names
Creator	WorkItemFactory
ConcreteCreator	UserTaskFactory, ServiceTaskFactory, ApprovalTaskFactory, ScriptTaskFactory, ParallelGroupFactory
Product	WorkElement
ConcreteProduct	UserTask, ServiceTask, ApprovalTask, ScriptTask, ParallelGroup

Collaboration: When an instance starts, the WorkflowEngine goes through each work-item spec in the WorkflowDefinition and hands it to the matching WorkItemFactory. The factory does the shared setup, such as linking the new element to the instance, while its concrete subclass decides which element to create, for example a UserTask or a ParallelGroup. The engine only receives a WorkElement back and never names a concrete class.

Variation: The kinds of work an instance contains differ between processes (meal tasks here, laboratory tests or approvals elsewhere), and new kinds appear over time. Only the creator hierarchy has to grow when this happens.

Why is it appropriate: The factory is the instance-creation side of the definition/instance split. It gives every instance its own freshly built objects from the shared definition, so changing one customer's address or cancelling their order cannot affect another order.

3. Composite

Problem: Work is nested: an instance may contain a single item or a group of items (in our scenario, a single meal or a full-day bundle), and a later item may start only when every item in the group before it is complete. The engine should treat an item and a group uniformly without type checks

Participants:

GoF Participants	Class Names
Component	WorkElement
Leaf	Task (UserTask, ServiceTask, ApprovalTask,ScriptTask)
Composite	WorkGroup (ParallelGroup)
Client	WorkflowEngine

Collaboration: The WorkflowEngine treats every WorkElement the same way and does not know whether it is a single Task or a WorkGroup. A ParallelGroup passes the engine's request on to its children, and it counts as complete only when all of them are complete.

Variation: The shape of an instance varies from a single task to a bundle of up to 4 tasks. In other workflows, work can nest more deeply as stages and sub-processes with parallel children.

Why is it appropriate: The Composite represents the nested work structure of a workflow. It lets the engine start, track and complete one meal or a whole bundle through one interface, so the packaging dependency does not need type checks or if/else chains.

4. State

Problem: A workflow instance’s valid actions depend on where it is in its lifecycle. Without a controlled lifecycle, every operation would have to check the status itself.

Participants:

GoF Participants	Class Names
State	InstanceState
ConcreteState	CreatedState, RunningState, SuspendedState, CompletedState, CancelledState, FailedState
Context	WorkflowInstance

Collaboration: The WorkflowInstance hands every request it receives to its current InstanceState and does not decide anything itself. Each ConcreteState either carries out the request and switches the instance to the next state. CreatedState moves the instance to RunningState when it is started. RunningState moves it on to CompletedState when all work is done, to FailedState when an issue stays unresolved, to SuspendedState when it is paused, or to CancelledState when cancelling is allowed.

Variation: The statuses an order passes through, and the actions each one permits, differ between processes. A workflow with an approval stage, for instance, would add states and change which transitions are valid.

Why is it appropriate: The State pattern gives the running instance a controlled lifecycle. Every operation is checked by the current state, so invalid actions such as cancelling an order in transit are rejected in one place instead of through status flags and switch statements.

5. Strategy

Problem: The delivery step must choose one of three couriers based on the customer's area, and other organisations would assign work by role or workload instead.

Participants:

GoF Participants	Class Names
Strategy	AssignmentStrategy
ConcreteStrategy	RoleMatchingStrategy, LeastLoadedStrategy, AttributeMatchStrategy
Context	AssignmentService

Collaboration: When a work item becomes available, the AssignmentService asks the AssignmentStrategy named in the definition to choose a participant from the candidates. AttributeMatchStrategy compares a field in the workflow data, such as an area, with the attributes of each candidate and picks the one that matches, and the AssignmentService then gives the item to that courier. Another workflow could name a different strategy without the AssignmentService changing.

Variation: How work is assigned differs between organisations and even between work items, whether by role, by workload or by a rule on workflow data such as delivery area. Each of these rules is a separate algorithm that can be swapped without touching the tasks.

Why is it appropriate: The Strategy is the routing and assignment decision point of the system. It keeps the question "who does this work" out of the task classes, so changing the rule, or choosing a different courier-selection approach, does not require rewriting any work item.

6. Chain of Responsibility

Problem: When ingredients run out or a delivery fails, the issue must climb from kitchen staff to supervisor to branch manager until someone resolves it, and nobody should need to know who handles what.

Participants:

GoF Participants	Class Names
Handler	EscalationHandler
ConcreteHandler	FrontlineHandler, SupervisorHandler, ManagerHandler
Client	WorkflowEngine

Collaboration: When a task reports an Issue, the WorkflowEngine passes it to the first FrontlineHandler, the lowest level in the chain. Each handler either resolves the issue that is within its authority or passes it to the next one (SupervisorHandler, then ManagerHandler). If the ManagerHandler cannot resolve it, the engine marks the instance as failed.

Variation: The number of escalation levels, their order, and which kinds of issue each level can resolve differ between organisations and between types of problem.

Why is it appropriate: This pattern is the exception-handling side of the system. The code that raises an issue does not need to know who resolves it. The engine only needs to know the first handler in the chain.

7. Observer

Problem: When packaging finishes, several unrelated things must happen: customer support is alerted, an SMS goes out, the audit log is written, and the next step is made available. The core model shouldn't know about any of them.

Participants:

GoF Participants	Class Names
Subject	WorkflowEventPublisher
ConcreteSubject	WorkflowInstance
Observer	WorkflowListener
ConcreteObserver	AuditLogger, NextItemActivator, NotificationListener, DashboardListener

Collaboration: The listeners register with the WorkflowInstance when it is created. When a task finishes, the instance publishes an event to all of them without knowing what they do. NotificationListener messages the concerned people, DashboardListener updates a live view of a workflow status, AuditLogger adds the event to the history, and NextItemActivator makes the next work item available, which can also move the instance on to its next state. The listeners work independently and do not know about each other.

Variation: Which parts of the system care about a given event, and what they do in response, differ between workflows and organisations. The same event might trigger an SMS today and an email or a warehouse update tomorrow.

Why is it appropriate: The Observer decouples the core model from everything that reacts to it. The model publishes events without knowing who receives them, so a new notification or integration only needs a new listener to be attached.

8. Adapter

Problem: TIVIDY must call external systems whose interfaces it did not design (in our scenario, the three couriers expose a web API, a legacy interface and a message queue), but a service task must work with any of them.

Participants:

GoF Participants	Class Names
Adaptee	RestApiClient, LegacySystemApi, MessageQueueClient
Target	ExternalServiceConnector
Adapter	RestApiAdapter, LegacySystemAdapter, MessagingAdapter
Client	ServiceTask

Collaboration: The ServiceTask works only with the ExternalServiceConnector interface, using the service chosen by the Strategy. The matching adapter holds that system's client, translates TIVIDY's request into its own format, and converts the response back into something TIVIDY understands.

Variation: Each courier exposes a different interface, data format and status vocabulary, and the set of external systems can grow or change over time.

Why is it appropriate: The Adapter is the integration boundary of the system. The engine and ServiceTask depend only on ExternalServiceConnector, so foreign systems stay outside the core and adding a fourth courier means writing one new adapter.

9. Decorator

Problem: A work element sometimes needs extra behaviour wrapped around it, such as applying a discount coupon to the order cost, writing audit entries or raising its priority. These extras must be combinable in any order without a separate subclass for every combination, and without making the task a different kind of task.

Participants:

GoF Participants	Class Names
Component	WorkElement
ConcreteComponent	Task (UserTask, ServiceTask, ApprovalTask, ScriptTask), ParallelGroup
Decorator	WorkElementDecorator
ConcreteDecorator	PayloadDecorator, AuditedDecorator, PriorityDecorator

Collaboration: Each decorator shares the WorkElement interface and wraps another WorkElement. For example, PayloadDecorator wraps a ParallelGroup and applies its configured rule to the group's data before the value reaches the engine. Decorators can be wrapped around each other, such as AuditedDecorator around PayloadDecorator around the group, and each passes everything else straight through to the element inside it.

Variation: Which extras apply to an item (one or several data rules, auditing, priority) differs per order and per workflow.

Why is it appropriate: The Decorator lets cross-cutting behaviour be attached to tasks and bundles without modifying them or creating a subclass per combination.

10. Memento

Problem: If the customer cancels or changes a coupon or address, the order must return to its previous state cleanly, and the history of what happened must not be erased.

Participants:

GoF Participants	Class Names
Originator	WorkflowInstance
Memento	InstanceSnapshot
Caretaker	RollbackManager

Collaboration: Before a risky change, the RollbackManager asks the WorkflowInstance to save a copy of its current state, and it stores the resulting InstanceSnapshot without reading it. On rollback, the manager gives the latest snapshot back to the instance, which restores itself from it. The AuditLogger then records the rollback, so the history only grows.

Variation: What needs to be captured (payload, states, assignments) and how many levels of undo are kept can differ between workflows.

Why is it appropriate: The Memento is the undo and rollback side of the system. Restoring from a snapshot returns the order to a known-good point while the history only grows, so the record stays accurate.

11. Template Method

Problem: Every work item follows the same lifecycle when it runs: check that it is ready, do the work, record the outcome, and notify the interested listeners. The "do the work" step is completely different for each kind of task and without a shared skeleton, each task kind would repeat the same steps.

Participants:

GoF Participants	Class Names
AbstractClass	Task
ConcreteClass	UserTask, ServiceTask, ApprovalTask, ScriptTask

Collaboration: The WorkflowEngine runs a Task without knowing which kind it is. The Task fixes the order

of the steps: it first checks that it is ready, then carries out the work, then records the outcome and notifies the listeners. Only the work step is left to the subclass. A UserTask waits for its assigned participant to finish, a ServiceTask passes the work to an external service through its connector, an ApprovalTask waits for an approval decision, and a ScriptTask runs a configured rule inside TIVIDY.

Variation: How the work itself gets done differs between kinds of task, and new kinds (a timed task, a scripted task) can appear, while the order of the surrounding steps stays fixed.

Why is it appropriate: It guarantees that every task is checked, recorded in the audit history and announced in exactly the same way, and it means the engine never has to branch on the kind of task.

12. Command

Problem: Actions on a workflow, such as starting it, cancelling it, changing its data or reassigning an item, need to be queued, logged and sometimes undone. If these actions are just direct method calls, the history can only show that something changed and they cannot be queued.

Participants:

GoF Participants	Class Names
Command	WorkflowCommand
ConcreteCommand	StartInstanceCommand, CancelInstanceCommand, ChangeDataCommand, ReassignItemCommand
Invoker	CommandInvoker
Receiver	WorkflowInstance
Client	TividyFacade

Collaboration: The TividyFacade receives a client's request and wraps it in a command object that carries everything needed to carry it out, including the WorkflowInstance it applies to. The facade passes the command to the CommandInvoker, which holds it until its turn and then runs it. For commands that change an instance, the invoker first has the RollbackManager save a restore point, so the command and its restore point are kept together.

Variation: The kinds of request that clients and operators can make differ between processes and keep growing, for example rescheduling or adding data. Each new request is a new command, and the invoker and the instance do not change.

Why is it appropriate: The Command is the part of the system that treats a request as something that can be kept, scheduled and reviewed, and not as a call that is made and forgotten. It sits between the entry point and the instance, so every change to a running workflow passes through one place where it can be queued, recorded and, where it makes sense, reversed.

13. Iterator

Problem: The engine needs to step through a group's work items one at a time, for example to find the next available item, and the audit history needs to be read entry by entry for reports.

Participants:

GoF Participants	Class Names
Iterator	WorkIterator
ConcreteIterator	DepthFirstIterator, AvailableItemIterator, AuditEntryIterator
Aggregate	WorkIterable
ConcreteAggregate	ParallelGroup, AuditHistory

Collaboration: When the engine needs the work items of a group, it asks the group for an iterator and then moves through the items one by one, without seeing how the group stores them. The AvailableItemIterator skips items that are already complete or not yet available. The DepthFirstIterator

goes into nested groups so that every task is reached. The AuditHistory hands out an AuditEntryIterator so that reports can read the log in order without seeing its storage.

Variation: The order in which work is visited, and which items are of interest, depend on the question being asked and on how deeply a process nests. A new traversal is a new iterator, and the groups and the log stay as they are.

Why is it appropriate: The Iterator is how the rest of the system walks through the pieces of work in the Composite and through the audit history, without depending on how either is stored.

14. Facade

Problem: To start a workflow, the outside world would otherwise have to deal with the designer and builder, the engine, the rollback and command history, the assignment service and the audit log. This leaks the system's structure to every user of TIVIDY and makes it easy to call things in the wrong order.

Participants:

GoF Participants	Class Names
Facade	TividyFacade
Subsystem classes	WorkflowDesigner, WorkflowEngine, CommandInvoker, RollbackManager, AuditLogger, AssignmentService

Collaboration: The client talks only to the TividyFacade. When it wants a new workflow, the facade gets a definition from the designer, has the engine start an instance from it, and sets up the listeners. When the client asks to cancel or change workflow, the facade passes the request to the command invoker. The subsystem classes do the work and do not know the facade exists.

Variation: The set of simple operations that clients need (start, cancel, track, report) can change and grow without changing the subsystem classes behind them.

Why is it appropriate: Facade is the entry point of the system. It hides the many cooperating subsystems behind a few operations, so the rest of the application can use TIVIDY without knowing its structure, and the subsystems can be reorganised without affecting users.

Task 5: UML Class Diagram

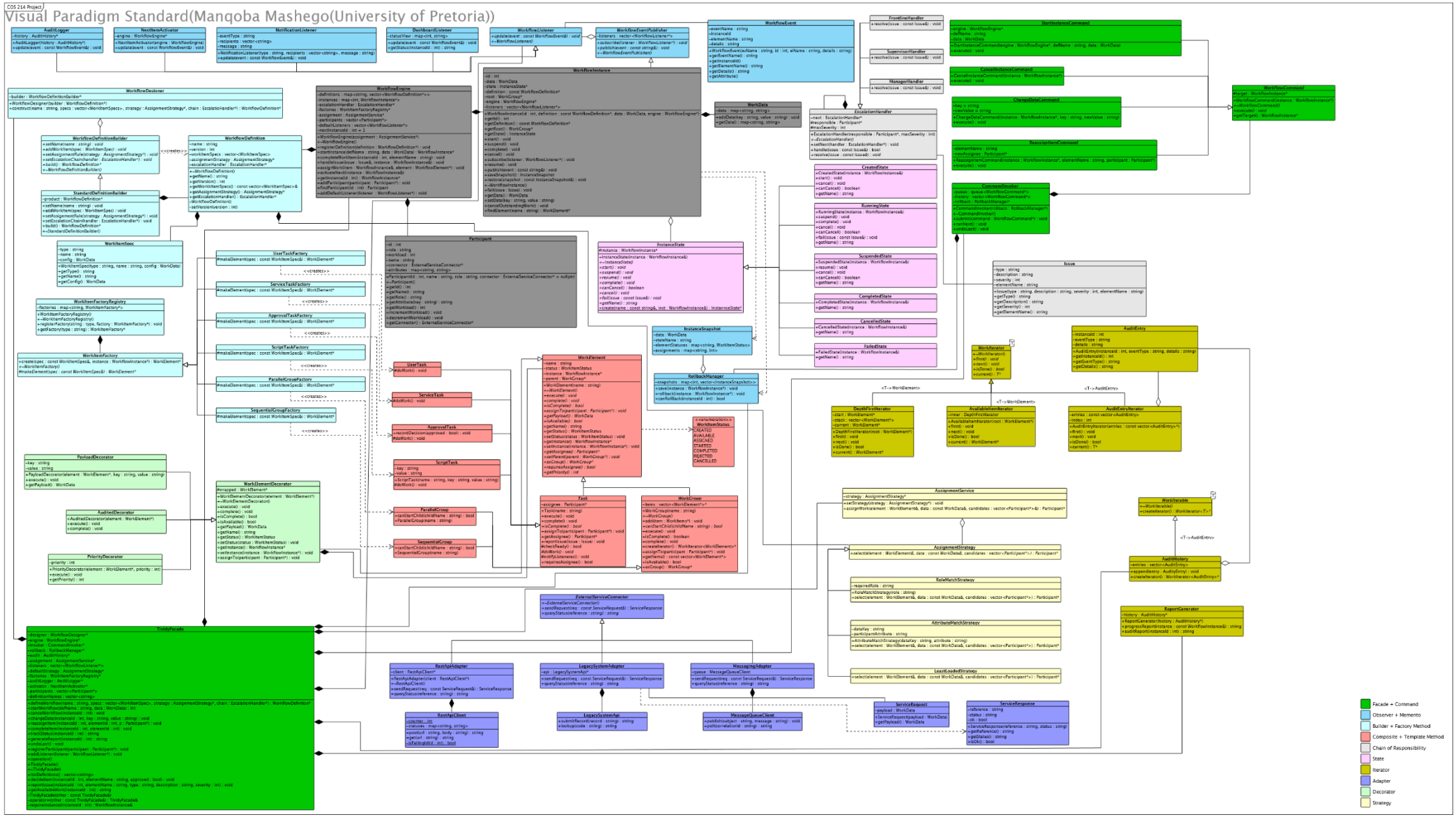


Figure 1: UML Class Diagram of 14 patterns

Task 6: Runtime Behaviour Diagrams

UML Sequence Diagram: Creating and starting a workflow instance

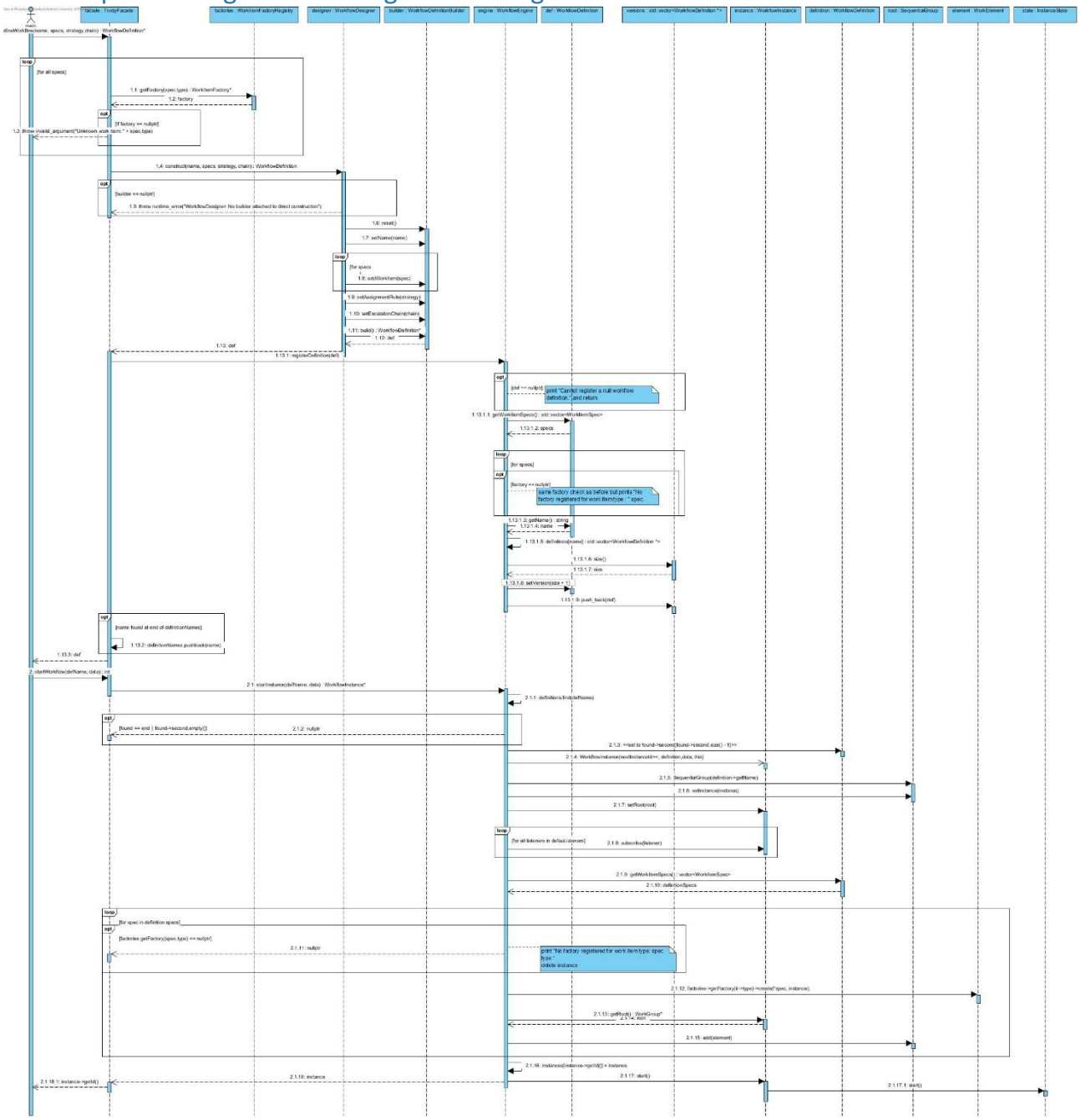


Figure 2: UML Sequence Diagram creating and starting a workflow instance

UML Sequence Diagram: Completing a workflow item and assigning the next one

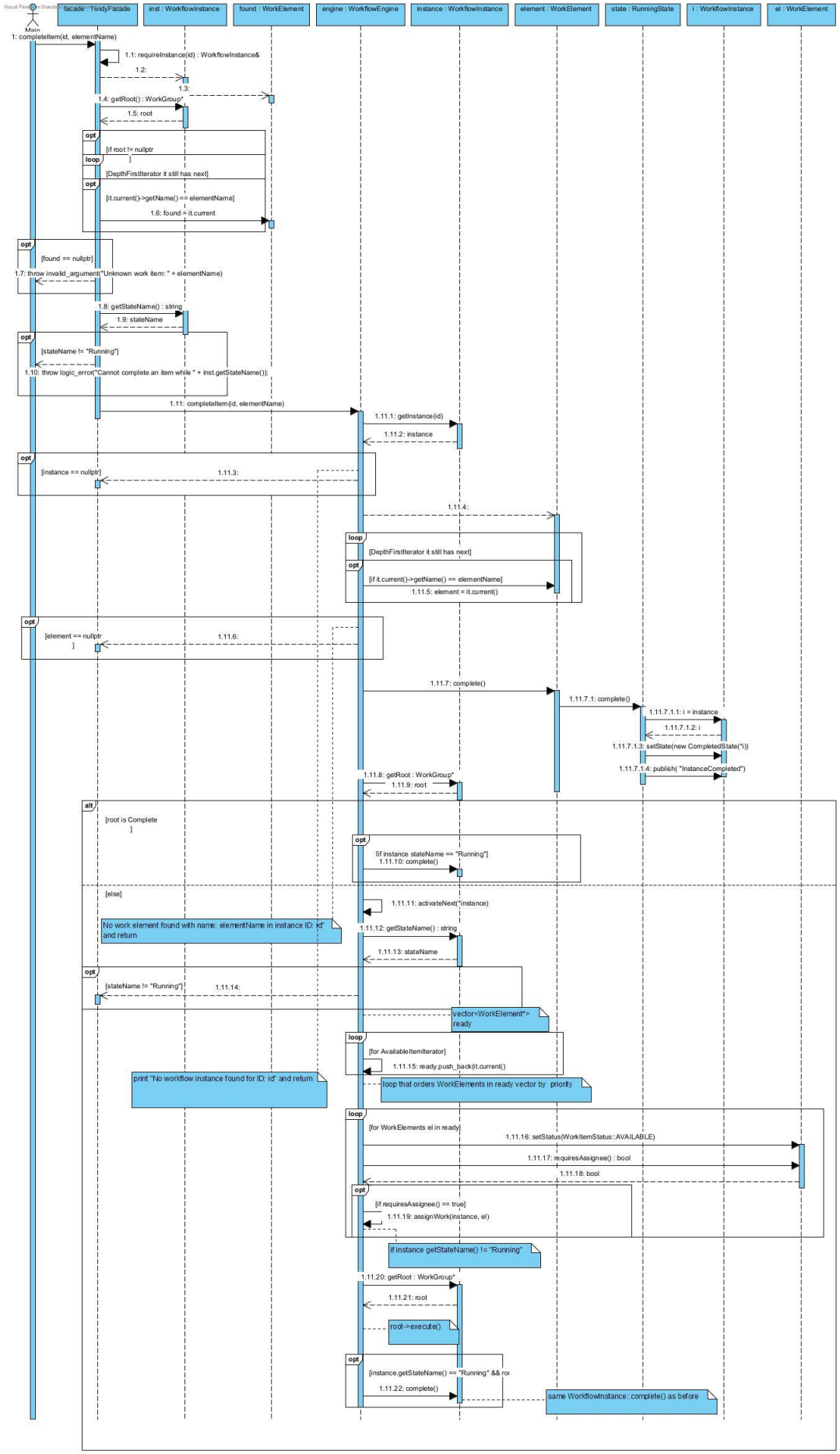


Figure 3: UML Sequence Diagram completing a workflow item

UML State Diagram: Workflow instance lifecycle

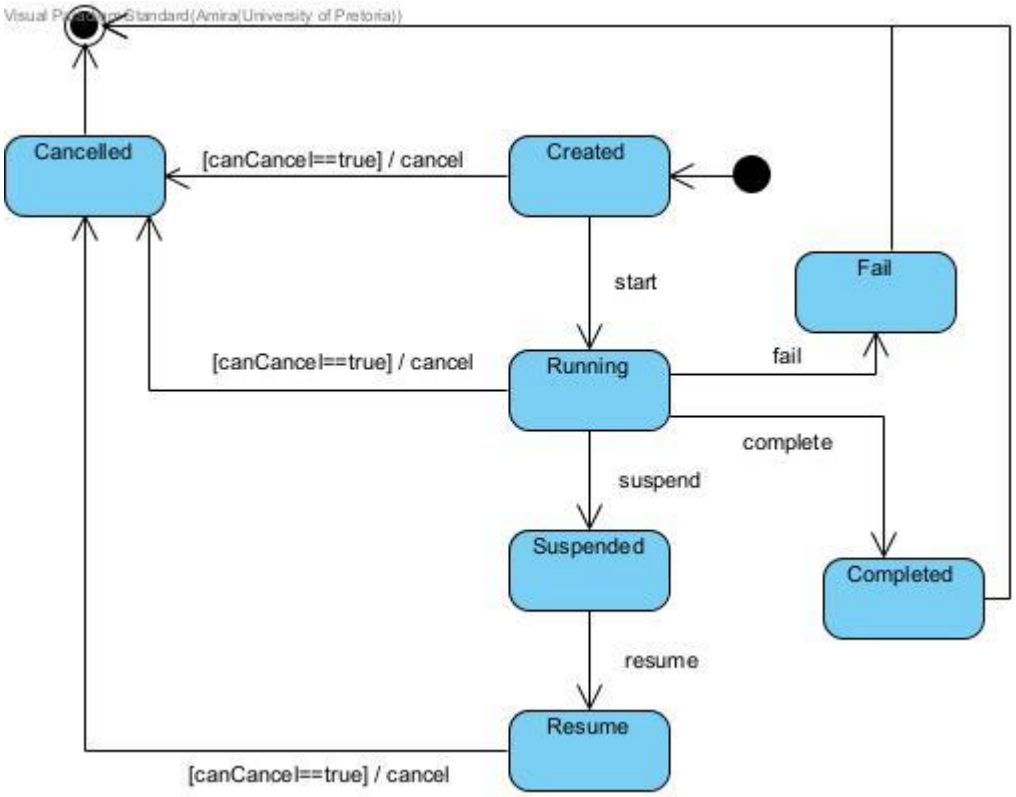


Figure 4: UML State Diagram of a workflow instance's lifecycle

Task 7

Decision 1: Generic class names

- **The problem or question being considered:** Whether the names in our UML class diagram were too specific to our chosen scenario, instead of describing a general workflow system that can work for multiple industries.
- **The original design or assumption:** A few of our initial class names used vocabulary closely linked to the scenario we use to test the workflow, such as KitchenStaffHandler, BranchManagerHandler and other classes named after a specific role.
- **The change that was made:** We discussed it as a group and with our tutor, and renamed the classes to describe their job in any workflow instead of one industry's vocabulary.
- **Why the change was made:** To make the workflow system more general, so that it can be reused for many different purposes. It also improves our level of abstraction.
- **Updates to the relevant diagrams:**
    - **Class diagram:** all classes in the table were renamed.
    - **Activity diagrams:** steps use the generic task types, and the roles are shown as the scenario's data.
    - **Sequence diagrams:** lifelines use the final class names, for example StandardDefinitionBuilder and WorkItemFactory

Decision 2: Validating definitions before use

- **The problem or question being considered:** Whether a bad definition could be registered and then fail or crash later.
- **The original design or assumption:** The Builder accepted any list of steps.
- **The change that was made:** The Builder refuses an empty list and duplicate step names, and it returns the caller's strategy and chain when it refuses. The facade rejects unknown step types when the workflow is defined. The engine numbers each definition's version when it registers it, and name lookup skips the hidden root group.
- **Why the change was made:** A definition has to be complete and valid before any instance runs from it, which is the reason for the Builder. A retried definition had crashed on a freed strategy and repeated names meant a workflow that never finished.
- **Updates to the relevant diagrams:**
    - **Sequence diagrams:** "Creating a workflow": add the validation step inside build().

Decision 3: Guarding the facade

- **The problem or question being considered:** Whether the single entry point could be used to break the system.
- **The original design or assumption:** The facade trusted the caller. It completed, decided or reassigned any step that existed, even one not yet reached or already finished, and reassigning a missing step crashed the program.
- **The change that was made:** The facade trusted the caller. It completed, decided or reassigned any step that existed, even one not yet reached or already finished, and reassigning a missing step crashed the program.
- **Why the change was made:** The facade is where invalid operations arrive, so it should refuse them instead of leaving the engine in a wrong state. The spec also asks for reasonable error handling for invalid transitions.
- **Updates to the relevant diagrams:**
    - **Class diagram:** The two new facade operations were added.

Implementation explanation

TIVIDY is written in C++11 and built with CMake. The engine is a static library, and the interactive text interface (TUI), the unit tests and the Docker images all link to it. The TUI is a menu driven with the arrow keys, and it talks only to TividyFacade.

How a workflow runs

1. The scenario setup passes each industry's steps, assignment rule, escalation chain and input data to TividyFacade::defineWorkflow. The WorkflowDesigner drives a StandardDefinitionBuilder, which validates the steps and produces a WorkflowDefinition. The engine registers it with a version number.
2. When the user starts a workflow, WorkflowEngine::startInstance creates a WorkflowInstance from the latest version. Each step description goes to the WorkItemFactoryRegistry, which picks a factory by type, and the factory creates the task. The tasks are added to a sequential root group, so each step becomes available only when the previous one is complete.
3. activateNext collects the ready steps with an AvailableItemIterator. For each one that needs a person, the AssignmentService asks the current AssignmentStrategy to choose a participant. The group then executes the step through Task::execute(). A user or approval task waits for a person, while a script or service task completes itself.
4. When a step completes, the instance publishes an event. The AuditLogger records it, and the NextItemActivator opens the next step. When every step is complete, the engine completes the instance.
5. A reported or rejected issue goes to the first EscalationHandler. The chain either resolves it, so the step is reset for rework (at most three times), or the instance moves to FailedState.
6. Before a command such as reassign or change data runs, the CommandInvoker asks the RollbackManager to save an InstanceSnapshot. Undo restores that snapshot.

The two industries:

Meal order delivery and fibre provisioning are each a definition, an assignment strategy, an escalation chain and input data, set up in the scenario code. Neither adds a class to the engine. Both are started from the same menu and run on the same TividyFacade, WorkflowEngine, WorkflowInstance, states, commands, iterators and observers.

Testing and delivery:

Each pattern has its own doctest file, and the facade tests run whole workflows end to end. We also wrote cross-pattern integration tests, described under the design decisions. A GitHub Actions pipeline builds and tests with g++ and clang++, and a Docker image runs the TUI.

Build, Run and Test

Prerequisites

A C++11 compiler (g++ or clang++), CMake 3.13 or newer and git. On Ubuntu or WSL: sudo apt install build-essential cmake git. The test framework, doctest, is included in tests/. Docker is optional.

Build and run

git clone https://github.com/ngeletech/COS214-PROJ.git
cd COS214-PROJ
cmake -S . -B build
cmake --build build
./build/tividy_app

Use the Up and Down arrow keys to move and Enter to select. Choose **Start Workflow**, then **MealOrderDelivery** or **FiberProvisioning**.

Run with Docker (optional)

docker compose build
docker compose run --rm tui

Run the tests

cd build
ctest --output-on-failure

Git and engineering evidence

Version control and review process

Each pattern pair worked on its own feature branch, such as shepherd/iterator or amira/memento, and opened a pull request into dev. A pull request was only merged once both CI jobs passed and was approved by at least 3 group members. Merging dev into a branch before opening its pull request kept conflicts small, mostly in the shared CMakeLists.txt

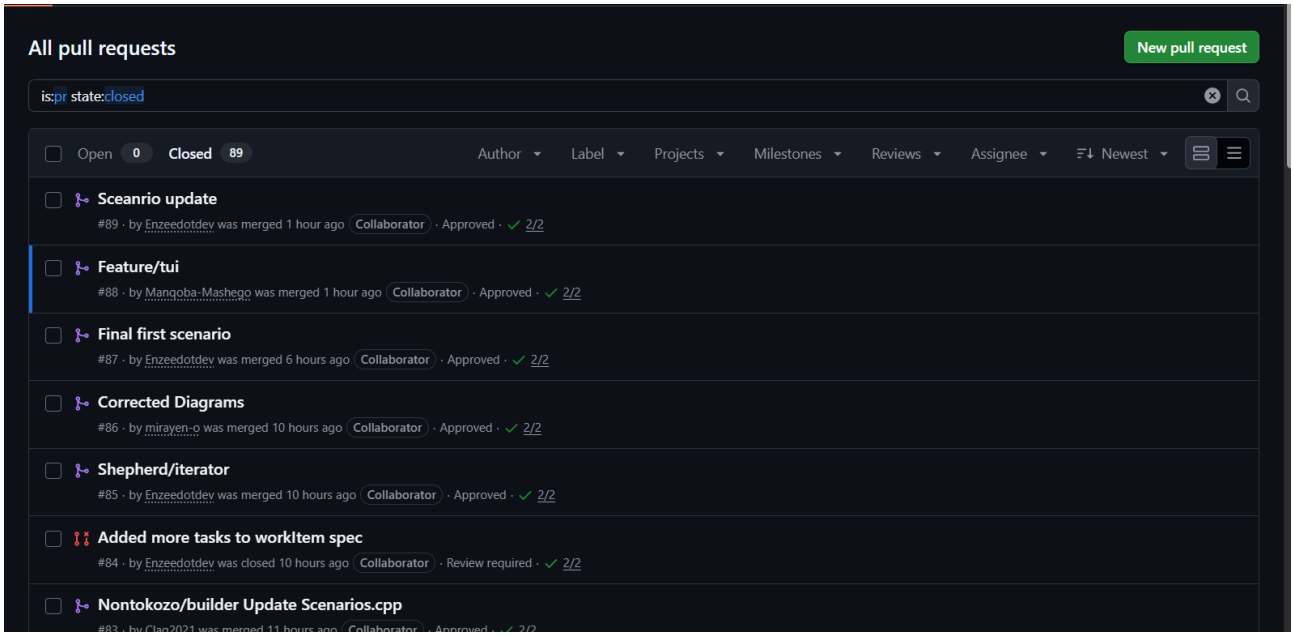


Figure 5: Closed Pull requests

Unit Testing

Every pattern has its own doctest file, and the facade tests run whole workflows end to end. The suite covers lifecycle transitions, assignment and routing, escalation, instance isolation, undo and invalid operations. Each member wrote tests for the patterns they own. We also wrote cross-pattern integration tests that checked how the patterns behave together.

```
t/COS214-PROJ/build$ ctest --output-on-failure
Test project /home/ngele_103_/COS214 Practicals/Project/COS214-PROJ/build
    Start 1: unit_tests
1/1 Test #1: unit_tests .....................   Passed    0.02 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.03 sec
```

Figure 6: All tests passing

CI/CD pipeline

A GitHub Actions workflow (.github/workflows/ci.yml) runs on every push and pull request to main and dev. It builds the project and runs the unit tests twice, once with g++ and once with clang++, using the warning flags -Wall -Wextra. The two jobs run independently, so a failure in one compiler never hides the result of the other. A pull request is only merged when both pass. This caught compiler-specific problems early and kept dev building for everyone, which mattered because seven people were changing a shared CMakeLists.txt. The result is visible on every pull request in the Checks tab.

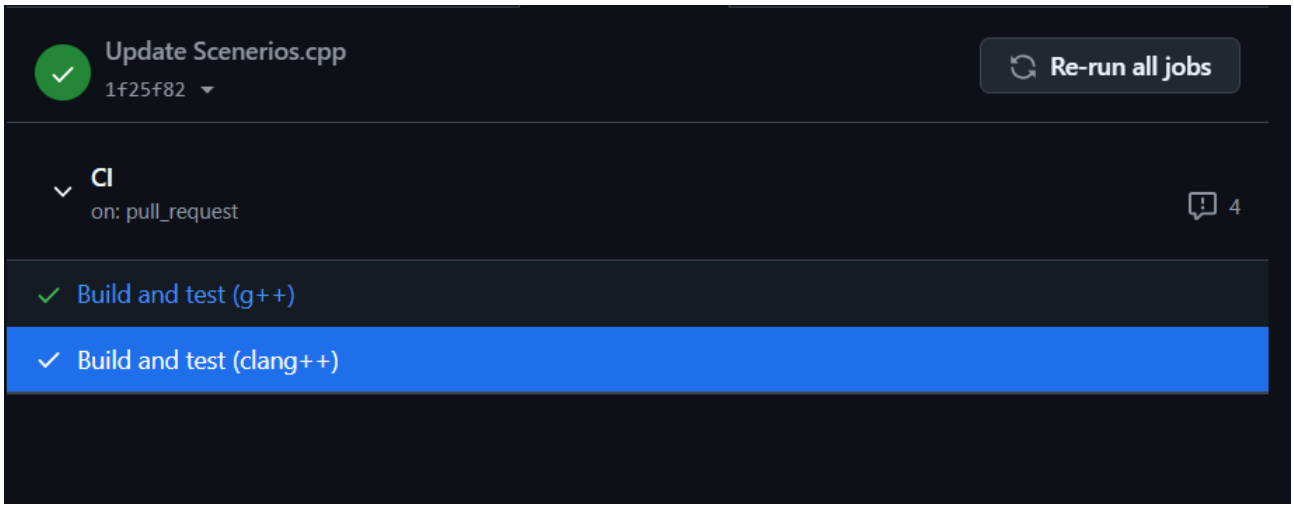


Figure 7: Ci g++ and clang++ on a pull request

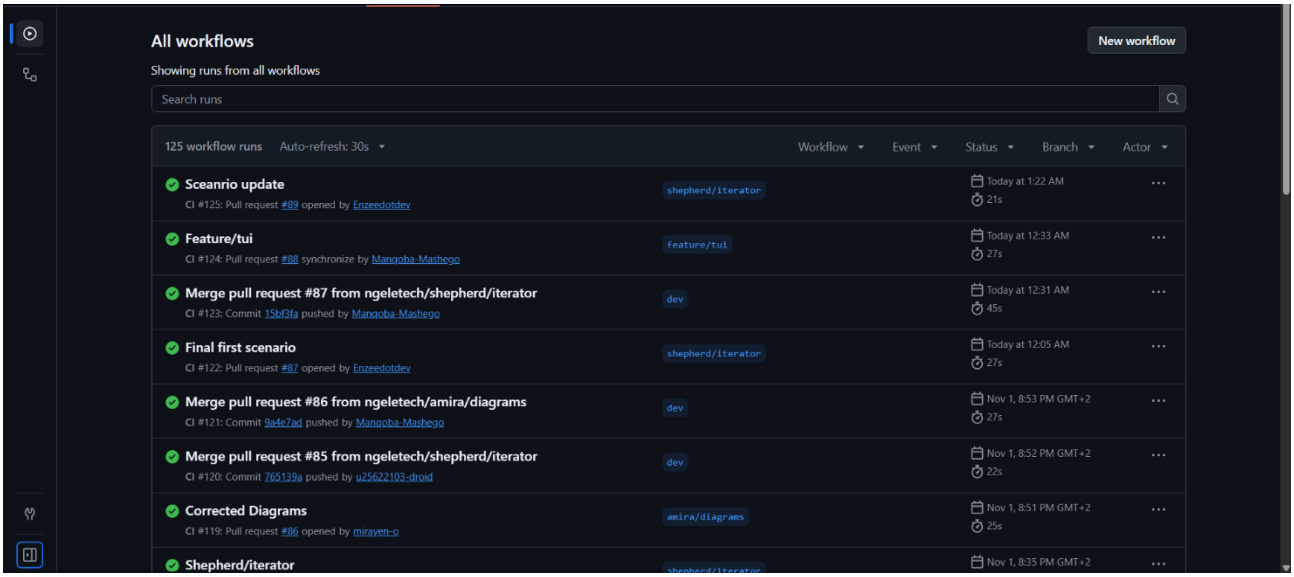


Figure 8: All workflows using CI pipeline

Docker

A Dockerfile and docker-compose.yml build the project on Ubuntu 22.04 and package it as two images. docker compose run --rm tui starts the interactive TUI, and docker compose run --rm tests runs the unit tests. This means anyone can run TIVIDY without installing a compiler or CMake, and the build is identical on every machine.

```
/COS214-PROJ$ docker compose run --rm tui
Container cos214-proj-tui-run-8ec67e8cb3c3 Creating
Container cos214-proj-tui-run-8ec67e8cb3c3 Created
[Setup]: Industry 1 (MealOrderDelivery) workflow & Observers registered!
[Setup]: Industry 2 (FiberProvisioning) workflow & Observers registered!

========================================
               TIVIDY
      WORKFLOW MANAGEMENT SYSTEM
========================================
MAIN MENU

  > 1. Start Workflow
    2. Active Workflows
    3. Exit

Use UP/DOWN arrows to navigate, ENTER to select.
```

Figure 9: Docker image running

Documentation and build quality

Every header has Doxygen comments, and the generated documentation is built with doxygen Doxyfile. Both CI compilers use the -Wall -Wextra warning flags. The README gives build, run and test instructions for both industries.

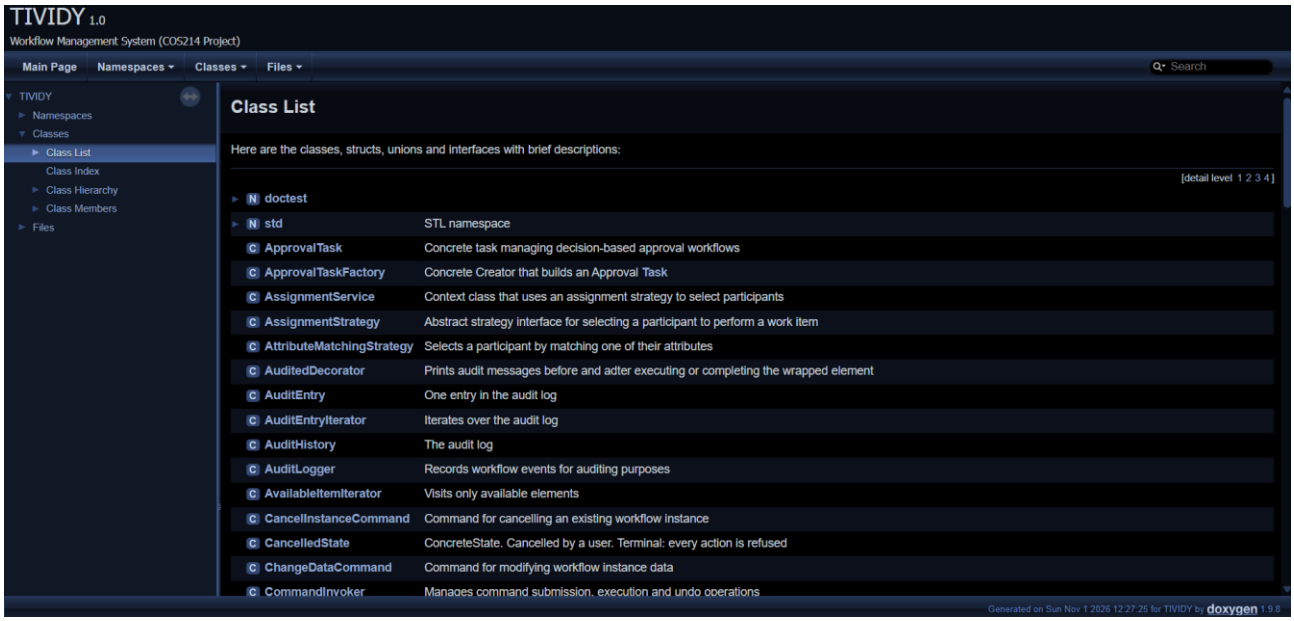


Figure 10: Doxyfile html

Contribution Statement

For this deliverable, tasks were allocated based on what each member felt was their strength and/or interest. The work was mostly divided by tasks with members then pushing the work from their branches to the main as each task finished allowing us to work in parallel. Once a member finished a section of the specification, they would notify the rest of us as teammates to verify what had been submitted and allow each other the opportunity to offer our opinions on anything that we felt needed to change. All members reviewed the integrated design to ensure the diagrams and documentation told a coherent story. The task allocation was as follows:

- **Task 1 :** Giselle
- **Task 2:** Nontokozo
- **Task 3:** Shepherd
- **Task 4:** Ngeletshedzo
- **Task 5:** Manqoba
- **Task 6:** Amira
- **Task 7/ Pdf:** Nare

Final Project Contributions

For the final project we built on the same way of working. Each pair kept the two patterns it owned in Practical 6, implemented them in C++11 on a feature branch, wrote its own unit tests, and opened a pull request into dev. A pull request was only merged once CI had built and tested it with both g++ and clang++.

Member	Implementation	Other Work
Ngeletshedzo	State, Facade	CI pipeline setup, documentation and ReadMe
Shepherd	Composite, Iterator	UML Activity Diagrams
Nontoko	Builder, Template Method	Scenario implementation
Amira	Factory Method, Memento	UML Sequence and State diagrams, debugger
Nare	Observer, Chain of Responsibility	TUI
Manqoba	Strategy, Command	UML Class Diagram and TUI
Giselle	Adapter, Decorator	Doxyfile and Docker

Work that was shared:

Engine and CMake: all members edited the shared CMakeLists.txt in their own labelled block and resolved conflicts by keeping each other's lines.

References

Camunda, n.d. *Versioning process definitions.* [Online]
Available at: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/
[Accessed 04 10 2026].
Object Management Group, 2011. *Business Process Model and Notation (BPMN).* [Online]
Available at: https://www.omg.org/spec/BPMN/2.0/PDF
[Accessed 04 10 2026].
Russell, N., van der Aalst, W., ter Hofstede, A. & Edmond, D., 2005. *Workflow resource patterns: identification, representation and tool support.* Berlin, Springer.
van der Aalst, W., 2013. Business process management: a comprehensive survey. *ISRN Software Engineering,* pp. 1-37.
van der Aalst, W., ter Hofstede, A., Kiepuszewski, B. & Barros, A., 2003. Workflow patterns. *Distributed and Parallel Databases,* 14(1), pp. 5-51.
Workflow Management Coalition, 1994. *The Workflow Reference Model.* [Online]
Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html
[Accessed 04 10 2026].